

企业技术迁移 · 实战指南

AI 驱动的 代码现代化

先定义终点与证据，再让智能体加速

基于 Anthropic 《How to prepare for AI-driven code modernization projects》

写得更快，不等于上线更快



变快的环节

智能体可以批量生成迁移变更



真正的瓶颈

验证、审批、发布仍按人工节奏运行

六步，把速度变成交付

01

定义目标

明确技术栈与行为

02

建立证书

自动检查“正确”

03

设定晋级策略

按风险决定评审

04

备齐条件

环境、测试、团队

05

打磨 workflow

从小范围迭代

06

试点后扩展

端到端证明可行

先选：究竟要改什么？

Uplift

版本升级

同一技术栈，升级运行时与依赖

行为保持不变

Transform

跨栈转换

换语言或框架，保留原有行为

以旧系统为参照

Reimagine

重新构想

新架构，同时修改业务行为

需要书面行为规格

第一步 / 补足上下文

先把旧系统“说清楚”

代码依赖与运行轨迹



抽取业务规则与边界条件



与用户、开发者、文档交叉核对

**“不知道旧系统做了什么，
就无法定义新系统对不对。”**

AI 可加速发现；业务访谈补足盲区。

每个变更都要带着证据

功能对等

原测试、输入输出对比、流量回放

非功能约束

性能界限、覆盖率、安全扫描

上线观察

暂存环境、错误率、延迟与告警

独立复核

不同上下文的对抗性审查

按风险分流，而不是逐行审完

低风险

证据充分

走标准化快速通道

需关注

出现标记

专家聚焦异常决策

关键路径

高影响变更

保留完整人工评审

开跑前，先把路修好

环境

可访问代码的独立主机、测试容量、生产遥测

团队

提前锁定领域专家与跨团队审批时间

代码与流水线

依赖图、兼容性校验、冻结与沟通计划

安全

最小权限、数据脱敏、变更和证据可追溯

小范围跑通，再规模化

输入给工作流

目标 + 证书 + 晋级策略 + 代码与文档

让并行的智能体处理小块迁移任务

选一个小分区



生成 → 验证 → 评审 → 合并



修正工作流，测算成本，再扩展

最后 / 带走一个原则

真正可复用的，不只是新代码

01

可复用的 workflow

02

可验证的证书

03

可接受的晋级策略

04

可追溯的证据链

先让组织准备好，再让 AI 跑起来。